

PHƯƠNG PHÁP THAM LAM (Greedy)

PGS.TS. Đặng Thị Oanh

Ngày 31 tháng 3 năm 2026

Chapter 9

Anany Levitin, **Introduction to the Design and Analysis of Algorithms**, ebook.

Review the Previous Lesson

- ➊ Vết cạn
- ➋ Quay lui
- ➌ Nhánh và Cạn

Nội dung

- 1 Giới thiệu về chiến lược Tham lam
- 2 Mô hình thuật toán
- 3 Một số bài toán điển hình
- 4 Thuật toán Prim tìm MST
- 5 Thuật toán Kruskal tìm MST
- 6 Thuật toán Dijkstra tìm đường đi ngắn nhất
- 7 Bài tập

Section 1

Giới thiệu về chiến lược Tham lam

Giới thiệu về chiến lược Tham lam

Giải thuật tham lam (Greedy algorithm)

là một thuật toán giải quyết một bài toán theo kiểu metaheuristic để tìm kiếm lựa chọn tối ưu địa phương ở mỗi bước đi với hy vọng tìm được tối ưu toàn cục.

Giới thiệu về chiến lược Tham lam

Giải thuật tham lam (Greedy algorithm)

là một thuật toán giải quyết một bài toán theo kiểu metaheuristic để tìm kiếm lựa chọn tối ưu địa phương ở mỗi bước đi với hy vọng tìm được tối ưu toàn cục.

Ví dụ 1: Bài toán đổi tiền

Có n tờ tiền (\$) với các mệnh giá $m_1, m_2, \dots, m_n$. Hãy tìm cách đổi $\$M$ để sao cho có số tờ tiền nhỏ nhất.

Giới thiệu về chiến lược Tham lam

Giải thuật tham lam (Greedy algorithm)

là một thuật toán giải quyết một bài toán theo kiểu metaheuristic để tìm kiếm lựa chọn tối ưu địa phương ở mỗi bước đi với hy vọng tìm được tối ưu toàn cục.

Ví dụ 1: Bài toán đổi tiền

Có n Tờ tiền (\$) với các mệnh giá $m_1, m_2, \dots, m_n$. Hãy tìm cách đổi $\$M$ để sao cho có số tờ tiền nhỏ nhất.

Áp dụng chiến thuật tham lam:

- 1 Sắp xếp các Tờ tiền theo mệnh giá giảm dần;
- 2 Chọn Tờ tiền có mệnh giá lớn nhất, sao cho đủ với số tiền còn lại.

Giới thiệu về chiến lược Tham lam

Giải thuật tham lam (Greedy algorithm)

là một thuật toán giải quyết một bài toán theo kiểu metaheuristic để tìm kiếm lựa chọn tối ưu địa phương ở mỗi bước đi với hy vọng tìm được tối ưu toàn cục.

Ví dụ 1: Bài toán đổi tiền

Có n Tờ tiền (\$) với các mệnh giá $m_1, m_2, \dots, m_n$. Hãy tìm cách đổi $\$M$ để sao cho có số tờ tiền nhỏ nhất.

Áp dụng chiến thuật tham lam:

- 1 Sắp xếp các Tờ tiền theo mệnh giá giảm dần;
- 2 Chọn Tờ tiền có mệnh giá lớn nhất, sao cho đủ với số tiền còn lại.

Cho các tờ tiền có mệnh giá \$1, \$2, \$5, \$20, \$50. Hãy tìm cách đổi \$97 để sao cho số tờ tiền nhỏ nhất?

$$\begin{aligned}\Rightarrow \$97 &= \$50 + 2 \times \$20 + \$5 + \$2 \\ &= 5 \text{ Tờ tiền.}\end{aligned}$$

Ví dụ 2: Bài toán xếp Ba lô

Có n loại đồ vật, $x_1, x_2, \dots, x_n$. Mỗi đồ vật x_j có một giá trị p_j và một trọng lượng w_j . Trọng lượng tối đa của Ba lô là C . Hãy tìm cách xếp các đồ vật vào Ba lô sao cho giá trị của Ba lô đạt lớn nhất.

Ví dụ 2: Bài toán xếp Ba lô

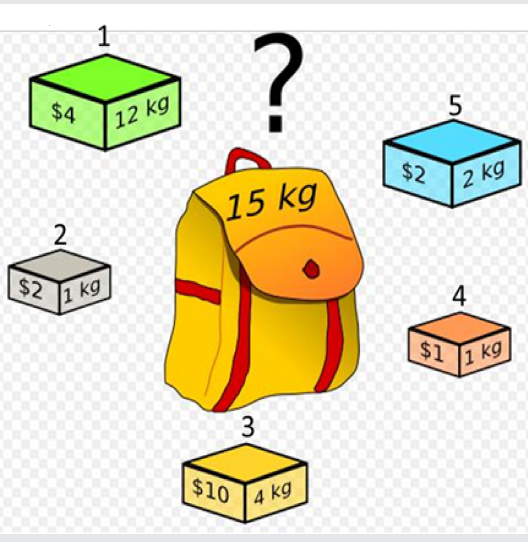
Có n loại đồ vật, $x_1, x_2, \dots, x_n$. Mỗi đồ vật x_j có một giá trị p_j và một trọng lượng w_j . Trọng lượng tối đa của Ba lô là C . Hãy tìm cách xếp các đồ vật vào Ba lô sao cho giá trị của Ba lô đạt lớn nhất.

Áp dụng chiến thuật tham lam:

- ➊ Tính đơn giá cho các đồ vật (giá trị/trọng lượng);
- ➋ Xét các đồ vật theo thứ tự đơn giá từ lớn đến nhỏ;
- ➌ Với mỗi đồ vật được xét, lấy một số lượng tối đa mà trọng lượng còn lại của ba lô cho phép;
- ➍ Xác định trọng lượng còn lại của ba lô và quay lại Bước 3 cho đến khi không còn có thể chọn được đồ vật nào nữa.

Giới thiệu về chiến lược Tham lam

Ba lô có trọng lượng là 15 kg và 5 loại đồ vật với trọng lượng và giá trị tương ứng được cho như sau:



- Tính đơn giá

Đồ vật	x_1	x_2	x_3	x_4	x_5
w (kg)	12	1	4	1	2
p (\$)	4	2	10	1	2
p_j/w_j	4/12	2	2.5	1	1

- Xếp theo đơn giá giảm dần

Đồ vật	x_3	x_2	x_4	x_5	x_1
w (kg)	4	1	1	2	12
p (\$)	10	2	1	2	4
p_j/w_j	2.5	2	1	1	4/12

$$\Rightarrow 15 \text{ kg} = x_3 \times 4 + x_2 \times 1$$

$$\text{Giá trị} = x_3 \times \$10 + x_2 \times \$2 = \$36.$$

Giới thiệu về chiến lược Tham lam

Câu hỏi: Chiến lược tham lam luôn cho lời giải tối ưu hay không?

Trả lời: "KHÔNG"

Ví dụ: Cho các tờ tiền có mệnh giá \$1, \$6, \$10. Hãy tìm cách đổi \$55 để sao cho số tờ tiền là ít nhất?

Greedy: $\$55 = 5 \times \$10 + 5 \times \$1 \Rightarrow 10$ Tờ tiền.

Not greedy:

$$\$55 = 4 \times \$10 + 2 \times \$6 + 3 \times \$1 \Rightarrow 9 \text{ Tờ tiền.}$$

$$\$55 = 3 \times \$10 + 4 \times \$6 + 1 \times \$1 \Rightarrow 8 \text{ Tờ tiền.}$$

Giới thiệu về chiến lược Tham lam

Câu hỏi: Chiến lược tham lam luôn cho lời giải tối ưu hay không?

Trả lời: "KHÔNG"

Ví dụ: Cho các tờ tiền có mệnh giá \$1, \$6, \$10. Hãy tìm cách đổi \$55 để sao cho số tờ tiền là ít nhất?

Greedy: $\$55 = 5 \times \$10 + 5 \times \$1 \Rightarrow 10$ Tờ tiền.

Not greedy:

$$\$55 = 4 \times \$10 + 2 \times \$6 + 3 \times \$1 \Rightarrow 9 \text{ Tờ tiền.}$$

$$\$55 = 3 \times \$10 + 4 \times \$6 + 1 \times \$1 \Rightarrow 8 \text{ Tờ tiền.}$$

Câu hỏi: Chiến lược tham lam luôn cho lời giải?

Trả lời: KHÔNG.

Ví dụ: Cho các tờ tiền có mệnh giá \$2, \$5, \$20, \$50. Hãy tìm cách đổi \$96 để sao cho số tờ tiền là ít nhất?

Greedy: $\Rightarrow \$96 = 1 \times \$50 + 2 \times \$20 + \$5 + ? \Rightarrow$ Có tìm được lời giải?

Not greedy:

$$\$96 = 1 \times \$50 + 2 \times \$20 + 3 \times \$2 \Rightarrow 6 \text{ Tờ tiền.}$$

Giới thiệu về chiến lược Tham lam

05 thành phần cốt lõi của chiến thuật tham lam

- ➊ Tập hợp ứng viên (candidate), để từ đó tạo lời giải;
- ➋ Hàm lựa chọn (chọn ứng viên tốt nhất để bổ sung vào lời giải);
- ➌ Hàm khả thi (feasibility) (kiểm tra điều kiện, ứng viên có/không được dùng để xây dựng lời giải);
- ➍ Hàm mục tiêu (giá trị lời giải), ấn định giá trị lời giải hoặc một lời giải chưa hoàn chỉnh;
- ➎ Hàm đánh giá (xác định khi nào kết thúc).

Giới thiệu về chiến lược Tham lam

05 thành phần cốt lõi của chiến thuật tham lam

- 1 Tập hợp ứng viên (candidate), để từ đó tạo lời giải;
- 2 Hàm lựa chọn (chọn ứng viên tốt nhất để bổ sung vào lời giải);
- 3 Hàm khả thi (feasibility) (kiểm tra điều kiện, ứng viên có/không được dùng để xây dựng lời giải);
- 4 Hàm mục tiêu (giá trị lời giải), ấn định giá trị lời giải hoặc một lời giải chưa hoàn chỉnh;
- 5 Hàm đánh giá (xác định khi nào kết thúc).

02 thành phần quyết định nhất tới hiệu quả của chiến lược

1 Tính chất lựa chọn tham lam:

- Chọn giải pháp tốt nhất ở thời điểm hiện tại và BT con được nảy sinh từ lựa chọn này. Lựa chọn có thể phụ thuộc vào các lựa chọn trước đó, nhưng không phụ thuộc vào bất kỳ lựa chọn nào đó trong tương lai hay phụ thuộc vào lời giải của các BT con;
- Giải thuật tham lam *quyết định sớm*, thay đổi đường đi thuật toán theo *quyết định đó* và **không bao giờ xét lại các quyết định cũ**;
- Với một số BT, **đây có thể là một thuật toán không chính xác**.

- ### 2 Cấu trúc con tối ưu:
- Một BT được gọi là "có cấu trúc tối ưu", nếu một lời giải tối ưu của BT con chứa lời giải tối ưu của BT lớn hơn.

Giới thiệu về chiến lược Tham lam

Đặc điểm và ưu/nhược điểm

- **Đặc điểm chính:** nhanh, đơn giản, không quay lui hay xem xét các lựa chọn tương lai, nhưng không phải lúc nào cũng tìm được đáp án tối ưu.
- **Ưu điểm:** Thuật toán chạy nhanh, dễ lập trình.
- **Nhược điểm:** Dễ bỏ qua lời giải tối ưu, nên thường chỉ áp dụng cho những BT đặc thù.

Section 2

Mô hình thuật toán

Mô hình thuật toán Tham lam

Quy trình thực hiện

- ➊ **Xác định cấu trúc:** Chọn tiêu chí tham lam (ví dụ: lấy đồ vật có giá trị cao nhất trước).
- ➋ **Sắp xếp:** Sắp xếp các lựa chọn dựa trên tiêu chí.
- ➌ **Lựa chọn:** Chọn phần tử "tốt nhất" hiện tại.
- ➍ **Kiểm tra:** Kiểm tra xem lựa chọn đó có khả thi không (thỏa mãn điều kiện).
- ➎ **Lặp lại:** Lặp lại quy trình (từ Bước 3) cho đến khi giải xong bài toán.

Để thuật toán tham lam đúng, cần chứng minh rằng sự lựa chọn tốt nhất tại mỗi bước sẽ dẫn đến tối ưu toàn cục.

Mô hình thuật toán Tham lam

The Greedy Template

- Input: A (problem domain – Tập các đối tượng của bài toán)
- Output: S (solution).

$S = \{x_1, x_2, \dots, x_n\}; x_i \in A$ such $f(S)$ is optimal

- Functions:
 - Chọn phần tử "tốt nhất" tại mỗi giai đoạn: $x = \text{BestSelect}(A)$.
 - Kiểm tra phần tử 'x' có thể trong lời giải hay không: $\text{Acceptable}(S, x)$.
 - Tích hợp (Integrate) phần tử 'x' vào tập lời giải S : $\text{Integrate}(S, x)$.
 - Kiểm tra sự đầy đủ của lời giải (kết thúc): $\text{IsSolution}(S)$.

Mô hình thuật toán Tham lam

The Greedy Template

- Input: A (problem domain – Tập các đối tượng của bài toán)
- Output: S (solution).

$$S = \{x_1, x_2, \dots, x_n\}; x_i \in A \text{ such } f(S) \text{ is optimal}$$

- Functions:
 - Chọn phần từ "tốt nhất" tại mỗi giai đoạn: $x = \text{BestSelect}(A)$.
 - Kiểm tra phần tử ' x ' có thể trong lời giải hay không: $\text{Acceptable}(S, x)$.
 - Tích hợp (Integrate) phần tử ' x ' vào tập lời giải S : $\text{Integrate}(S, x)$.
 - Kiểm tra sự đầy đủ của lời giải (kết thúc): $\text{IsSolution}(S)$.

The Greedy Diagram

Greedy(A) $\equiv$ //Optimal solution by greedy on domain A

$S = \emptyset$;

while ($A \neq \emptyset$ and $\neg \text{IsSolution}(S)$)

$x = \text{BestSelect}(A)$;

$A = A \setminus \{x\}$;

if ($\text{Acceptable}(S, x)$)

$\text{Integrate}(S, x)$;

endwhile;

return S ;

End.

Section 3

Một số bài toán điển hình

Một số bài toán điển hình

- ❶ **Bài toán trả tiền lẻ (Change coins):** Lấy các đồng tiền có mệnh giá lớn nhất có thể cho đến khi đủ số tiền cần trả.
- ❷ **Bài toán balo (Fractional Knapsack):** Lấy các vật phẩm có tỉ lệ giá trị/trọng lượng cao nhất.
- ❸ **Activity Selection Problem - Lập lịch tối đa hóa khoảng t.**
- ❹ **Minimum Spanning Tree Problem (MSTP) - Cây bao trùm tối thiểu.**

Một số bài toán điển hình

Change coins - Đổi tiền lẻ Template

Input: $X = \{x_1, x_2, \dots, x_n\}$; T *// x_i : mệnh giá, T : Số tiền*
Output: $S = \{k_1, k_2, \dots, k_n\}$ | *// k_i số đồng xu x_i*

Diagram

```
ChangeMoney ( $A, T$ )  $\equiv$   
    Sắp  $X$  giảm dần theo mệnh giá  
    for ( $i = 1..n$ )  $k_i = 0$ ;  
     $i = 1$ ;  
    while ( $i \leq n$  and  $T > 0$ )  
         $k_i = T/x_i$ ;  
         $T = T - k_i \times x_i$ ;  
         $i = i + 1$ ;  
    endwhile;  
    if ( $T > 0$ ) return -1;      //No solution  
    else return  $\{k_1, k_2, \dots, k_n\}$ 
```

End.

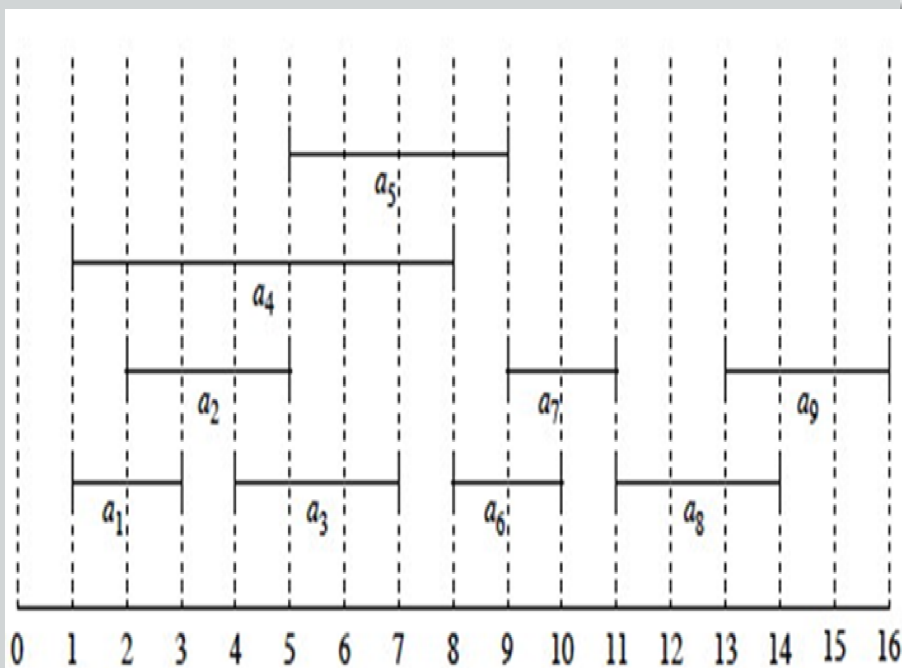
Một số bài toán điển hình

Activity Selection Problem - Lập lịch tối đa hóa khoảng thời gian

- n hoạt động sử dụng một nguồn lực duy nhất (machine, room...).
- Hoạt động i diễn ra trong khoảng từ s_i (start) đến f_i (final) ($s_i < f_i$) và có toàn quyền sử dụng nguồn lực trong thời gian đã lên lịch.
- Tìm chuỗi hoạt động tương thích dài nhất mà không có bất kỳ sự chồng chéo nào?

Minh họa

a_i	s_i	f_i
a_1	1	3
a_2	2	5
a_3	4	7
a_4	1	8
a_5	5	9
a_6	8	10
a_7	9	11
a_8	11	14
a_9	13	16



Activity Selection Problem

• Problem specification

- Input: $A = \{a_i = (s_i, f_i); s_i < f_i; i = 1 \dots n\}$
- Output: $S = \{a'_1; a'_2; \dots a'_m | a'_i \in A; f'_i \leq s'_{i+1}; m\text{-max}\}$

• Áp dụng chiến lược tham lam cho bài toán tối ưu

Tại mỗi bước, chọn hoạt động kết thúc sớm nhất, ta sẽ có cơ hội chọn các hoạt động phù hợp nhất tiếp theo.

• Template

- Xếp các hoạt động theo thứ tự thời gian kết thúc tăng dần ($f_i < f_{i+1}, i = \overline{1, n-1}$);
- Tại bước i
 - Các hoạt động theo lịch trình $a'_1; a'_2; \dots; a'_{i-1}$
 - Lập biểu a_i bằng cách chọn $a_i = (s_i, f_i) \in A_i = A \setminus \{a'_1; a'_2; \dots a'_{i-1}\}$, sao cho $s_i \geq f'_{i-1}$ và f_i nhỏ nhất trong A_i .
- Bắt đầu: $a'_1 = a_1$

Một số bài toán điển hình

Activity Selection Problem - Diagram

ActivitySelector(A) $\equiv$

$S = \{a_1\}$

$j = 1;$

for ($i = 2 \dots n$)

if ($s_i \geq f_j$)

$S = S \cup \{a_i\};$

$j = i;$

endif;

endfor;

return S;

End.

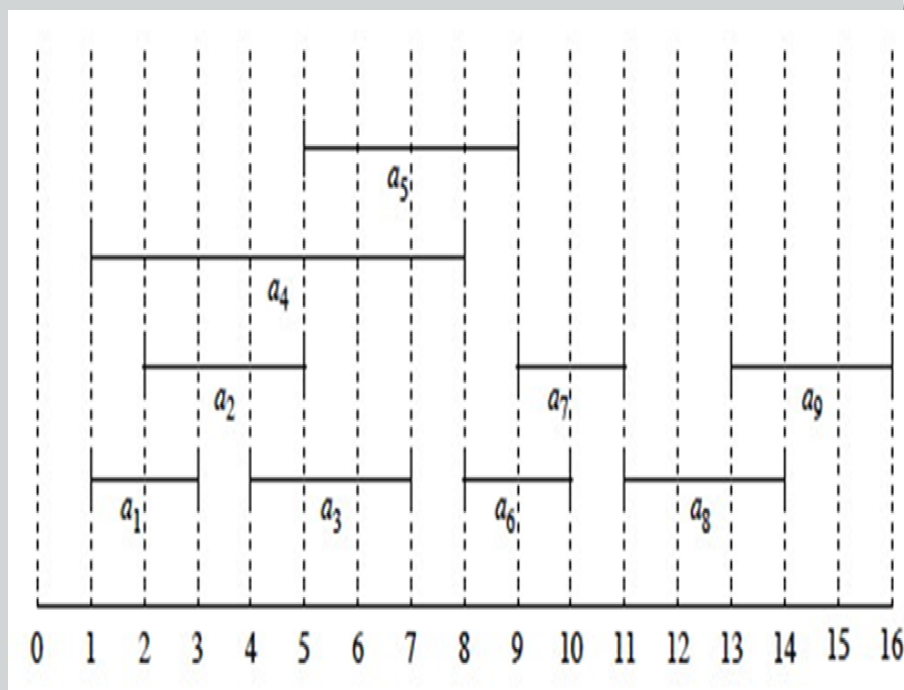
Where: $A = \{a_1, a_2, \dots, a_n\}$ được xếp theo thứ tự tăng dần của thời gian kết thúc ($f_i < f_{i+1}$)

Một số bài toán điển hình

Minh họa ứng dụng chiến lược tham lam

Ví dụ

a_i	s_i	f_i
a_1	1	3
a_2	2	5
a_3	4	7
a_4	1	8
a_5	5	9
a_6	8	10
a_7	9	11
a_8	11	14
a_9	13	16

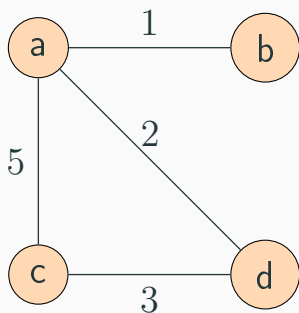


$$S = \{a_1, a_3, a_6, a_8\}$$

Một số bài toán điển hình

Minimum Spanning Tree Problem - Cây bao trùm tối thiểu

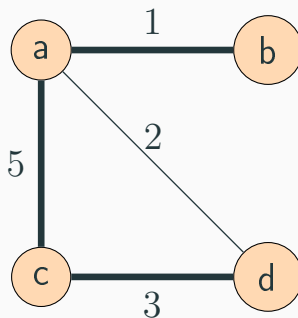
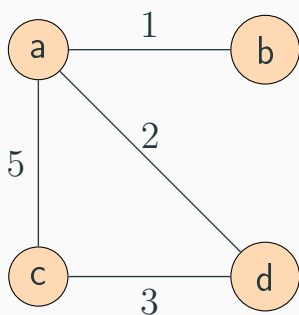
- Đồ thị $G(V, E)$: Tập đỉnh V và tập cạnh E có trọng số.
- Cây bao trùm (Spanning Tree): Đồ thị con liên thông, không có chu trình, chứa tất cả các đỉnh (n đỉnh, $n - 1$ cạnh).
- Cây bao trùm tối thiểu (MST): Cây bao trùm có tổng trọng số các cạnh nhỏ nhất.



Một số bài toán điển hình

Minimum Spanning Tree Problem - Cây bao trùm tối thiểu

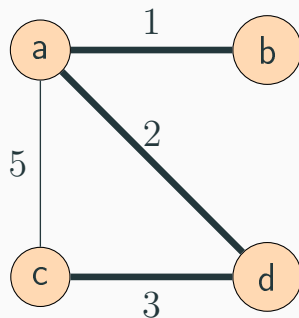
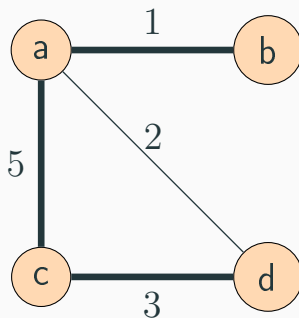
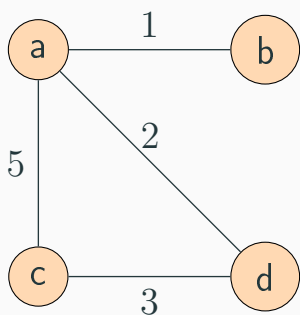
- Đồ thị $G(V, E)$: Tập đỉnh V và tập cạnh E có trọng số.
- Cây bao trùm (Spanning Tree): Đồ thị con liên thông, không có chu trình, chứa tất cả các đỉnh (n đỉnh, $n - 1$ cạnh).
- Cây bao trùm tối thiểu (MST): Cây bao trùm có tổng trọng số các cạnh nhỏ nhất.



Một số bài toán điển hình

Minimum Spanning Tree Problem - Cây bao trùm tối thiểu

- Đồ thị $G(V, E)$: Tập đỉnh V và tập cạnh E có trọng số.
- Cây bao trùm (Spanning Tree): Đồ thị con liên thông, không có chu trình, chứa tất cả các đỉnh (n đỉnh, $n - 1$ cạnh).
- Cây bao trùm tối thiểu (MST): Cây bao trùm có tổng trọng số các cạnh nhỏ nhất.



Section 4

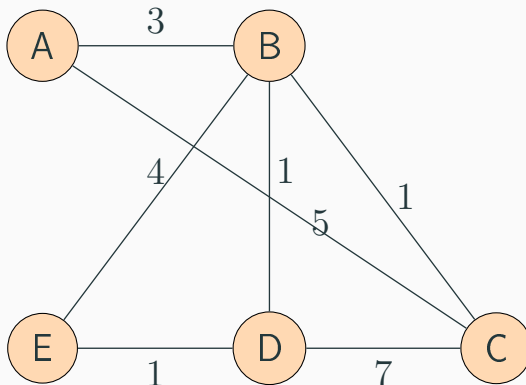
Thuật toán Prim tìm MST

Thuật toán Prim (Phát triển cây)

- **Ý tưởng:** Kết nạp từng đỉnh vào đồ thị theo tiêu chí: đỉnh được nạp vào tiếp theo phải chưa được nạp và gần nhất với các đỉnh đã được nạp vào đồ thị.
- **Các bước của thuật toán**
 - ➊ **Bước 1:** Khởi tạo cây T chỉ chứa 1 đỉnh bất kỳ (T_0).
 - ➋ **Bước 2 (Tham lam):** Tại mỗi bước, tìm cạnh nhỏ nhất kết nối một đỉnh trong cây T với một đỉnh nằm ngoài cây T (mở rộng cây T : $T_1, T_2, \dots, T_{n-1}$).
 - ➌ **Bước 3:** Thêm cạnh và đỉnh mới (xác định tại Bước 2) vào cây T .
 - ➍ **Bước 4:** Lặp lại cho đến khi tất cả các đỉnh đều được kết nối.

Ưu điểm: Hiệu quả với đồ thị dày (nhiều cạnh).

Thuật toán Prim tìm MST

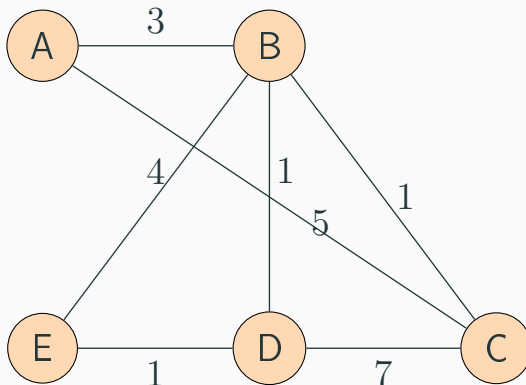


Ví dụ minh họa

Các Bước

- ❶ **Khởi tạo:** Chọn 1 đỉnh, giả sử chọn A. Tập đỉnh $V_T = \{A\}$, tập cạnh $E_T = \emptyset$.

Thuật toán Prim tìm MST

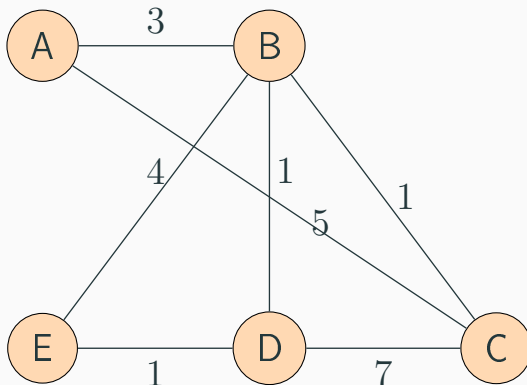


Ví dụ minh họa

Các Bước

- 1 Khởi tạo:** Chọn 1 đỉnh, giả sử chọn A. Tập đỉnh $V_T = \{A\}$, tập cạnh $E_T = \emptyset$.
- 2 Bước 1:** Tìm cạnh nhỏ nhất nối $\{A\}$ với $\{B, C, D, E\}$. Cạnh nhỏ nhất $A - B$ (trọng số 3)
 $\Rightarrow V_T = \{A, B\}$; $E_T = \{(A, B)\}$.

Thuật toán Prim tìm MST

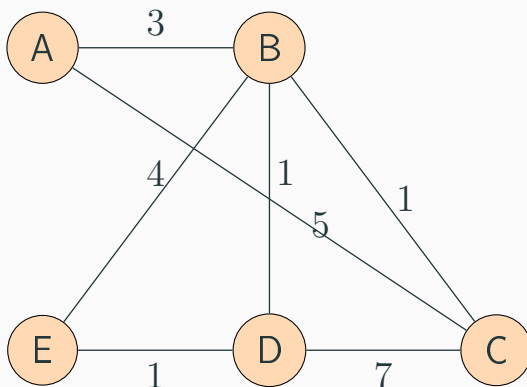


Ví dụ minh họa

Các Bước

- 1 Khởi tạo:** Chọn 1 đỉnh, giả sử chọn A. Tập đỉnh $V_T = \{A\}$, tập cạnh $E_T = \emptyset$.
- 2 Bước 1:** Tìm cạnh nhỏ nhất nối $\{A\}$ với $\{B, C, D, E\}$. Cạnh nhỏ nhất $A - B$ (trọng số 3)
 $\Rightarrow V_T = \{A, B\}$; $E_T = \{(A, B)\}$.
- 3 Bước 2:** Tìm cạnh nhỏ nhất nối $\{A, B\}$ với $\{C, D, E\}$. Các cạnh ứng viên $A - C(5)$, $B - C(1)$, $B - D(1)$, $B - E(4)$. Chọn $B - C \Rightarrow V_T = \{A, B, C\}$; $E_T = \{(A, B), (B, C)\}$.

Thuật toán Prim tìm MST

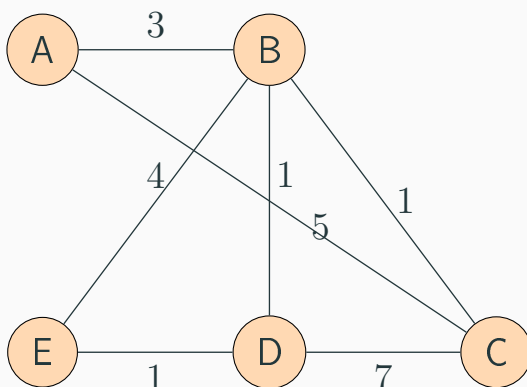


Ví dụ minh họa

Các Bước

- 1 Khởi tạo:** Chọn 1 đỉnh, giả sử chọn A. Tập đỉnh $V_T = \{A\}$, tập cạnh $E_T = \emptyset$.
- 2 Bước 1:** Tìm cạnh nhỏ nhất nối $\{A\}$ với $\{B, C, D, E\}$. Cạnh nhỏ nhất $A - B$ (trọng số 3)
 $\Rightarrow V_T = \{A, B\}$; $E_T = \{(A, B)\}$.
- 3 Bước 2:** Tìm cạnh nhỏ nhất nối $\{A, B\}$ với $\{C, D, E\}$. Các cạnh ứng viên $A - C(5)$, $B - C(1)$, $B - D(1)$, $B - E(4)$. Chọn $B - C \Rightarrow V_T = \{A, B, C\}$; $E_T = \{(A, B), (B, C)\}$.
- 4 Bước 3:** Tìm cạnh nhỏ nhất nối $\{A, B, C\}$ với $\{D, E\}$. Các cạnh ứng viên $B - D(1)$, $B - E(4)$, $C - D(7)$. Chọn $B - D \Rightarrow V_T = \{A, B, C, D\}$; $E_T = \{(A, B), (B, C), (B, D)\}$.

Thuật toán Prim tìm MST

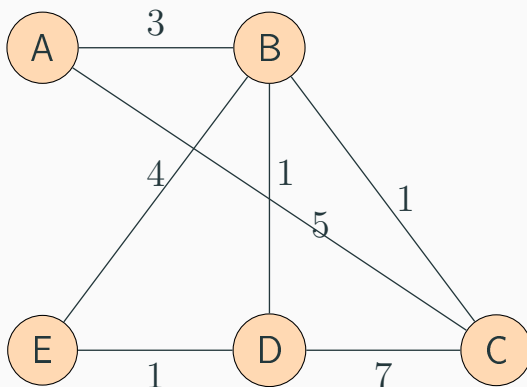


Ví dụ minh họa

Các Bước

- 1 Khởi tạo:** Chọn 1 đỉnh, giả sử chọn A. Tập đỉnh $V_T = \{A\}$, tập cạnh $E_T = \emptyset$.
- 2 Bước 1:** Tìm cạnh nhỏ nhất nối $\{A\}$ với $\{B, C, D, E\}$. Cạnh nhỏ nhất $A - B$ (trọng số 3) $\Rightarrow V_T = \{A, B\}$; $E_T = \{(A, B)\}$.
- 3 Bước 2:** Tìm cạnh nhỏ nhất nối $\{A, B\}$ với $\{C, D, E\}$. Các cạnh ứng viên $A - C(5)$, $B - C(1)$, $B - D(1)$, $B - E(4)$. Chọn $B - C \Rightarrow V_T = \{A, B, C\}$; $E_T = \{(A, B), (B, C)\}$.
- 4 Bước 3:** Tìm cạnh nhỏ nhất nối $\{A, B, C\}$ với $\{D, E\}$. Các cạnh ứng viên $B - D(1)$, $B - E(4)$, $C - D(7)$. Chọn $B - D \Rightarrow V_T = \{A, B, C, D\}$; $E_T = \{(A, B), (B, C), (B, D)\}$.
- 5 Bước 4:** Tìm cạnh nhỏ nhất nối $\{A, B, C, D\}$ với $\{E\}$. Các cạnh ứng viên $D - E(1)$, $B - E(4)$. Chọn $D - E \Rightarrow V_T = \{A, B, C, D, E\}$; $E_T = \{(A, B), (B, C), (B, D), (D, E)\}$.

Thuật toán Prim tìm MST



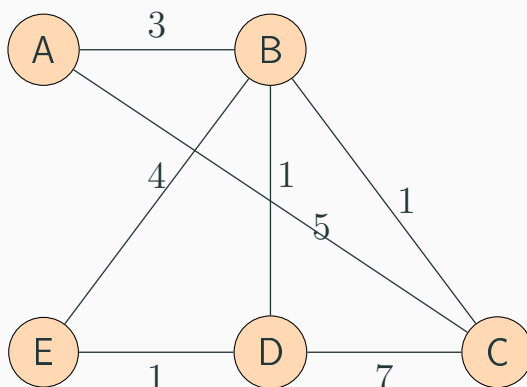
Ví dụ minh họa

Các Bước

- 1 Khởi tạo:** Chọn 1 đỉnh, giả sử chọn A. Tập đỉnh $V_T = \{A\}$, tập cạnh $E_T = \emptyset$.
- 2 Bước 1:** Tìm cạnh nhỏ nhất nối $\{A\}$ với $\{B, C, D, E\}$. Cạnh nhỏ nhất $A - B$ (trọng số 3)
 $\Rightarrow V_T = \{A, B\}$; $E_T = \{(A, B)\}$.
- 3 Bước 2:** Tìm cạnh nhỏ nhất nối $\{A, B\}$ với $\{C, D, E\}$. Các cạnh ứng viên $A - C(5)$, $B - C(1)$, $B - D(1)$, $B - E(4)$. Chọn $B - C \Rightarrow V_T = \{A, B, C\}$; $E_T = \{(A, B), (B, C)\}$.
- 4 Bước 3:** Tìm cạnh nhỏ nhất nối $\{A, B, C\}$ với $\{D, E\}$. Các cạnh ứng viên $B - D(1)$, $B - E(4)$, $C - D(7)$. Chọn $B - D \Rightarrow V_T = \{A, B, C, D\}$; $E_T = \{(A, B), (B, C), (B, D)\}$.
- 5 Bước 4:** Tìm cạnh nhỏ nhất nối $\{A, B, C, D\}$ với $\{E\}$. Các cạnh ứng viên $D - E(1)$, $B - E(4)$. Chọn $D - E \Rightarrow V_T = \{A, B, C, D, E\}$; $E_T = \{(A, B), (B, C), (B, D), (D, E)\}$.

Kết quả: MST gồm các cạnh: $(A-B)(3), (B-C)(1), (B-D)(1), (D,E)(1)$.

Thuật toán Prim tìm MST



Ví dụ minh họa

Các Bước

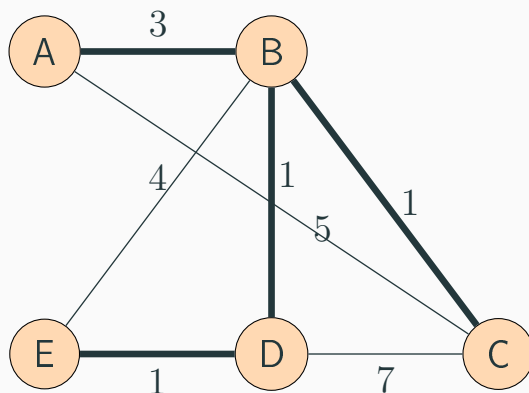
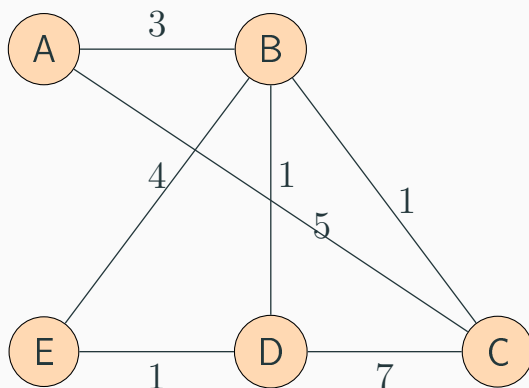
- 1 Khởi tạo:** Chọn 1 đỉnh, giả sử chọn A. Tập đỉnh $V_T = \{A\}$, tập cạnh $E_T = \emptyset$.
- 2 Bước 1:** Tìm cạnh nhỏ nhất nối $\{A\}$ với $\{B, C, D, E\}$. Cạnh nhỏ nhất $A - B$ (trọng số 3)
 $\Rightarrow V_T = \{A, B\}$; $E_T = \{(A, B)\}$.
- 3 Bước 2:** Tìm cạnh nhỏ nhất nối $\{A, B\}$ với $\{C, D, E\}$. Các cạnh ứng viên $A - C(5)$, $B - C(1)$, $B - D(1)$, $B - E(4)$. Chọn $B - C \Rightarrow V_T = \{A, B, C\}$; $E_T = \{(A, B), (B, C)\}$.
- 4 Bước 3:** Tìm cạnh nhỏ nhất nối $\{A, B, C\}$ với $\{D, E\}$. Các cạnh ứng viên $B - D(1)$, $B - E(4)$, $C - D(7)$. Chọn $B - D \Rightarrow V_T = \{A, B, C, D\}$; $E_T = \{(A, B), (B, C), (B, D)\}$.
- 5 Bước 4:** Tìm cạnh nhỏ nhất nối $\{A, B, C, D\}$ với $\{E\}$. Các cạnh ứng viên $D - E(1)$, $B - E(4)$. Chọn $D - E \Rightarrow V_T = \{A, B, C, D, E\}$; $E_T = \{(A, B), (B, C), (B, D), (D, E)\}$.

Kết quả: MST gồm các cạnh: $(A-B)(3), (B-C)(1), (B-D)(1), (D,E)(1)$.

Trọng số: $3 + 1 + 1 + 1 = 6$.

Thuật toán Prim tìm MST

Ví dụ minh họa



Các Bước

- Khởi tạo:** Chọn 1 đỉnh, giả sử chọn A. Tập đỉnh $V_T = \{A\}$, tập cạnh $E_T = \emptyset$.
- Bước 1:** Tìm cạnh nhỏ nhất nối $\{A\}$ với $\{B, C, D, E\}$. Cạnh nhỏ nhất $A - B$ (trọng số 3) $\Rightarrow V_T = \{A, B\}$; $E_T = \{(A, B)\}$.
- Bước 2:** Tìm cạnh nhỏ nhất nối $\{A, B\}$ với $\{C, D, E\}$. Các cạnh ứng viên $A - C(5)$, $B - C(1)$, $B - D(1)$, $B - E(4)$. Chọn $B - C \Rightarrow V_T = \{A, B, C\}$; $E_T = \{(A, B), (B, C)\}$.
- Bước 3:** Tìm cạnh nhỏ nhất nối $\{A, B, C\}$ với $\{D, E\}$. Các cạnh ứng viên $B - D(1)$, $B - E(4)$, $C - D(7)$. Chọn $B - D \Rightarrow V_T = \{A, B, C, D\}$; $E_T = \{(A, B), (B, C), (B, D)\}$.
- Bước 4:** Tìm cạnh nhỏ nhất nối $\{A, B, C, D\}$ với $\{E\}$. Các cạnh ứng viên $D - E(1)$, $B - E(4)$. Chọn $D - E \Rightarrow V_T = \{A, B, C, D, E\}$; $E_T = \{(A, B), (B, C), (B, D), (D, E)\}$.

Kết quả: MST gồm các cạnh: $(A-B)(3), (B-C)(1), (B-D)(1), (D,E)(1)$.

Trọng số: $3 + 1 + 1 + 1 = 6$.

Thuật toán Prim tìm MST

Input: $G = (V, E)$

Output: V_T, E_T tập các đỉnh, tập các cạnh MST của G .

Prim(G) $\equiv$

$V_T = \{v_0\};$

$E_T = \emptyset;$

for $i = 1 \dots \#V - 1$

1. Tìm cạnh nhỏ nhất $e^* = (v^*, u^*)$ thỏa mãn $v \in V_T$ and $u \in V \setminus V_T$;
2. $V_T = V_T \cup \{u^*\};$
 $E_T = E_T \cup \{e^*\}.$

endfor

return V_T, E_T ;

End.

Thuật toán Prim tìm MST

Tính đúng đắn của Thuật toán Prim

Sử dụng phương pháp quy nạp và phản chứng

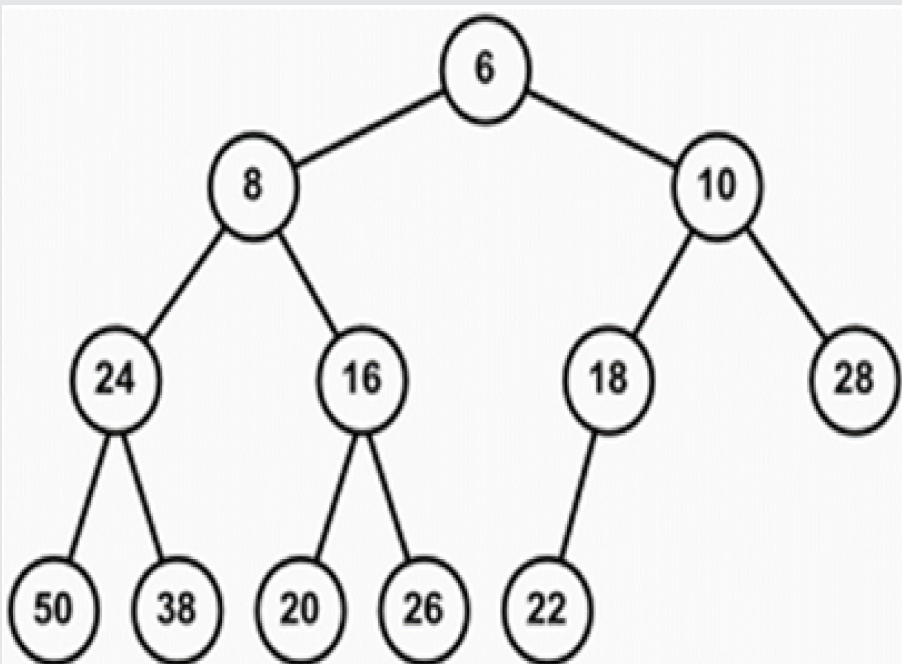
- **Chứng minh bằng quy nạp:** mỗi cây con T_i , $i = 0, \dots, n - 1$, sinh ra bằng thuật toán Prim là một cây bao trùm tối thiểu (MST). Giả sử T_{i-1} là một phần của MST thì T_i (sinh ra bằng cách thêm vào cạnh gần nhất) cũng là một phần của MST.
- **Phản chứng:** giả sử MST của G không chứa T_i , ta có cạnh $e_i = (v, u)$ là cạnh có trọng số nhỏ nhất nối một đỉnh thuộc T_{i-1} với 1 đỉnh không thuộc T_{i-1} và e_i không thuộc cây tối thiểu T (theo Prim).
Nếu thêm e_i vào T sẽ tạo ra một chu trình, chu trình sẽ chứa cạnh $e'_I = (v', u')$ nối một đỉnh $v' \in T_{i-1}$ với 1 đỉnh $u' \notin T_{i-1}$. Nếu xóa đi cạnh (v', u') của chu trình này sẽ nhận được một cây bao trùm khác chứa e_i có trọng số nhỏ hơn T . Như vậy trái với giả thiết MST không chứa $T_i \Rightarrow$ (đpcm).

Thuật toán Prim tìm MST

Độ phức tạp thời gian

Xác định đỉnh “gần nhất”:

- Sử dụng ma trận kề biểu diễn G : Duyệt đỉnh với độ phức tạp $O(V^2)$
- Sử dụng cây nhị phân đồng nhỏ nhất (min heap) và danh sách kề biểu diễn đồ thị: $O(E \log V)$



Section 5

Thuật toán Kruskal tìm MST

Thuật toán Kruskal tìm MST

Thuật toán Kruskal (Tham lam)

- **Ý tưởng:** Ban đầu mỗi đỉnh là một cây riêng biệt, ta tìm cây khung nhỏ nhất bằng cách duyệt các cạnh theo trọng số từ nhỏ đến lớn, rồi hợp nhất các cây lại với nhau.
- **Mô tả thuật toán** Tìm MST của đồ thị G
 - Khởi tạo rừng (forest) $F = \emptyset$ (tập hợp các cây), trong đó mỗi đỉnh của G tạo thành một cây riêng biệt.
 - Khởi tạo tập S chứa tất cả các cạnh của G .
 - Chừng nào S còn khác rỗng và F gồm hơn một cây
 - Xóa cạnh nhỏ nhất (giả sử cạnh e) trong S ;
 - Nếu e nối 2 cây khác nhau $\in F$: $F = F \cup e$ và hợp hai cây kề với nó làm một.

Khi thuật toán kết thúc, rừng (F) chỉ gồm đúng một cây và đó là một cây bao trùm nhỏ nhất của đồ thị G .

Thuật toán Kruskal tìm MST

Thuật toán Kruskal (Tham lam)

- **Mã giả:** Đồ thị $G = (X, E)$
 - ➊ **Bước 1:** Sắp xếp các cạnh của đồ thị G theo thứ tự trọng số tăng dần.
 - ➋ **Bước 2:** Khởi tạo $T := \emptyset$
 - ➌ **Bước 3:** Lần lượt lấy từng cạnh e thuộc danh sách đã sắp xếp. Nếu $T + \{e\}$ không chứa chu trình thì gán $T = T + \{e\}$.
 - ➍ **Bước 4:** Nếu T đủ $n - 1$ phần tử thì dừng, trái lại quay lại Bước 3.
- **Chú ý:**
 - **Trong quá trình xây dựng T :** các cạnh có thể không liên thông, lúc đó T chỉ là rừng (chưa trở thành cây).
 - **Khi thuật toán dừng:** Nếu T chưa đủ $n - 1$ cạnh, đồ thị G không liên thông (không có cây khung). Ngược lại, T là cây khung cần tìm.

Phương pháp này đảm bảo tổng trọng số cạnh là nhỏ nhất, phù hợp cho các bài toán tối ưu hóa mạng lưới.

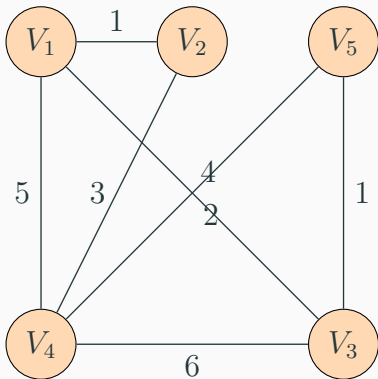
Ưu điểm: Hiệu quả với đồ thị thưa (ít cạnh).

Thuật toán Kruskal tìm MST

Ví dụ minh họa: Cho $G = (X, E)$, $X = \{V_1, V_2, V_3, V_4, V_5\}$,
 $E = \{(V_1, V_2; 1), (V_1, V_3; 2), (V_1, V_4; 5), (V_2, V_4; 3), (V_3, V_4; 6), (V_3, V_5; 1), (V_4, V_5; 4)\}$

Các Bước

- 1 Sắp xếp cạnh theo trọng số tăng dần
 $E = \{(V_1, V_2; 1), (V_3, V_5; 1), (V_1, V_3; 2), (V_2, V_4; 3), (V_4, V_5; 4), (V_1, V_4; 5), (V_3, V_4; 6)\}$

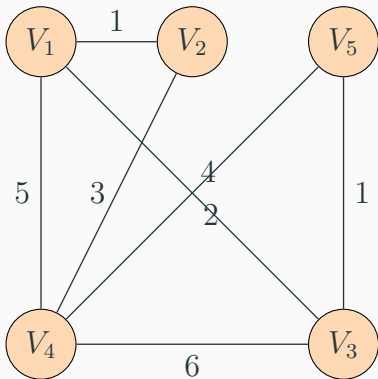


Thuật toán Kruskal tìm MST

Ví dụ minh họa: Cho $G = (X, E)$, $X = \{V_1, V_2, V_3, V_4, V_5\}$,
 $E = \{(V_1, V_2; 1), (V_1, V_3; 2), (V_1, V_4; 5), (V_2, V_4; 3), (V_3, V_4; 6), (V_3, V_5; 1), (V_4, V_5; 4)\}$

Các Bước

- ❶ Sắp xếp cạnh theo trọng số tăng dần
 $E = \{(V_1, V_2; 1), (V_3, V_5; 1), (V_1, V_3; 2), (V_2, V_4; 3), (V_4, V_5; 4), (V_1, V_4; 5), (V_3, V_4; 6)\}$
- ❷ Chọn cạnh (kèm kiểm tra chu trình)
 1. Chọn $(V_1, V_2; 1)$: Cạnh MST,
 $T = \{(V_1, V_2)\}$

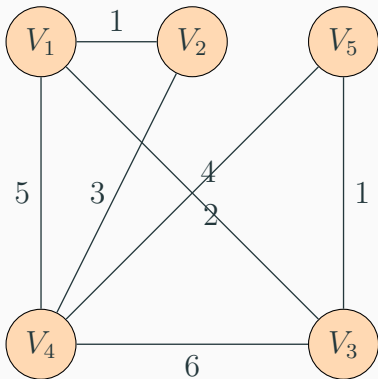


Thuật toán Kruskal tìm MST

Ví dụ minh họa: Cho $G = (X, E)$, $X = \{V_1, V_2, V_3, V_4, V_5\}$,
 $E = \{(V_1, V_2; 1), (V_1, V_3; 2), (V_1, V_4; 5), (V_2, V_4; 3), (V_3, V_4; 6), (V_3, V_5; 1), (V_4, V_5; 4)\}$

Các Bước

- ❶ Sắp xếp cạnh theo trọng số tăng dần
 $E = \{(V_1, V_2; 1), (V_3, V_5; 1), (V_1, V_3; 2), (V_2, V_4; 3), (V_4, V_5; 4), (V_1, V_4; 5), (V_3, V_4; 6)\}$
- ❷ Chọn cạnh (kèm kiểm tra chu trình)
 1. Chọn $(V_1, V_2; 1)$: Cạnh MST,
 $T = \{(V_1, V_2)\}$
 2. Chọn $(V_3, V_5; 1)$: Cạnh MST,
 $T = \{(V_1, V_2), (V_3, V_5)\}$

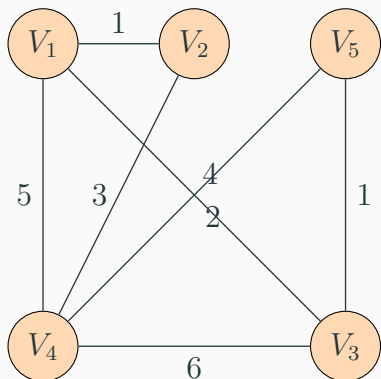


Thuật toán Kruskal tìm MST

Ví dụ minh họa: Cho $G = (X, E)$, $X = \{V_1, V_2, V_3, V_4, V_5\}$,
 $E = \{(V_1, V_2; 1), (V_1, V_3; 2), (V_1, V_4; 5), (V_2, V_4; 3), (V_3, V_4; 6), (V_3, V_5; 1), (V_4, V_5; 4)\}$

Các Bước

- ❶ Sắp xếp cạnh theo trọng số tăng dần
 $E = \{(V_1, V_2; 1), (V_3, V_5; 1), (V_1, V_3; 2), (V_2, V_4; 3), (V_4, V_5; 4), (V_1, V_4; 5), (V_3, V_4; 6)\}$
- ❷ Chọn cạnh (kèm kiểm tra chu trình)
 1. Chọn $(V_1, V_2; 1)$: Cạnh MST,
 $T = \{(V_1, V_2)\}$
 2. Chọn $(V_3, V_5; 1)$: Cạnh MST,
 $T = \{(V_1, V_2), (V_3, V_5)\}$
 3. Chọn $(V_1, V_3; 2)$: Cạnh MST,
 $T = \{(V_1, V_2), (V_3, V_5), (V_1, V_3)\}$

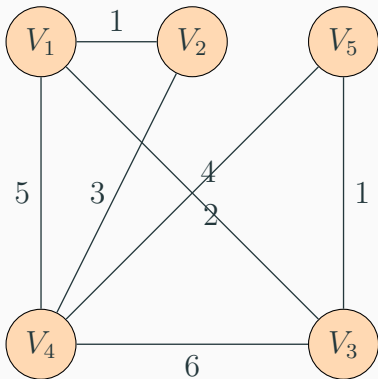


Thuật toán Kruskal tìm MST

Ví dụ minh họa: Cho $G = (X, E)$, $X = \{V_1, V_2, V_3, V_4, V_5\}$,
 $E = \{(V_1, V_2; 1), (V_1, V_3; 2), (V_1, V_4; 5), (V_2, V_4; 3), (V_3, V_4; 6), (V_3, V_5; 1), (V_4, V_5; 4)\}$

Các Bước

- ❶ Sắp xếp cạnh theo trọng số tăng dần
 $E = \{(V_1, V_2; 1), (V_3, V_5; 1), (V_1, V_3; 2), (V_2, V_4; 3), (V_4, V_5; 4), (V_1, V_4; 5), (V_3, V_4; 6)\}$
- ❷ Chọn cạnh (kèm kiểm tra chu trình)
 1. Chọn $(V_1, V_2; 1)$: Cạnh MST,
 $T = \{(V_1, V_2)\}$
 2. Chọn $(V_3, V_5; 1)$: Cạnh MST,
 $T = \{(V_1, V_2), (V_3, V_5)\}$
 3. Chọn $(V_1, V_3; 2)$: Cạnh MST,
 $T = \{(V_1, V_2), (V_3, V_5), (V_1, V_3)\}$
 4. Chọn $(V_2, V_4; 3)$: Cạnh MST,
 $T = \{(V_1, V_2), (V_3, V_5), (V_1, V_3), (V_2, V_4)\}$

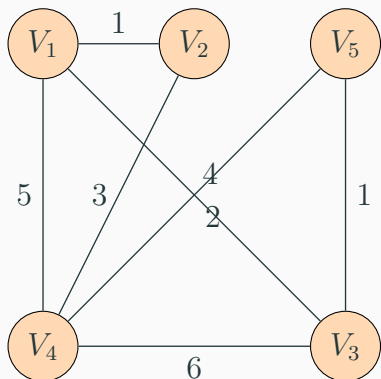


Thuật toán Kruskal tìm MST

Ví dụ minh họa: Cho $G = (X, E)$, $X = \{V_1, V_2, V_3, V_4, V_5\}$,
 $E = \{(V_1, V_2; 1), (V_1, V_3; 2), (V_1, V_4; 5), (V_2, V_4; 3), (V_3, V_4; 6), (V_3, V_5; 1), (V_4, V_5; 4)\}$

Các Bước

- ❶ Sắp xếp cạnh theo trọng số tăng dần
 $E = \{(V_1, V_2; 1), (V_3, V_5; 1), (V_1, V_3; 2), (V_2, V_4; 3), (V_4, V_5; 4), (V_1, V_4; 5), (V_3, V_4; 6)\}$
- ❷ Chọn cạnh (kèm kiểm tra chu trình)
 1. Chọn $(V_1, V_2; 1)$: Cạnh MST,
 $T = \{(V_1, V_2)\}$
 2. Chọn $(V_3, V_5; 1)$: Cạnh MST,
 $T = \{(V_1, V_2), (V_3, V_5)\}$
 3. Chọn $(V_1, V_3; 2)$: Cạnh MST,
 $T = \{(V_1, V_2), (V_3, V_5), (V_1, V_3)\}$
 4. Chọn $(V_2, V_4; 3)$: Cạnh MST,
 $T = \{(V_1, V_2), (V_3, V_5), (V_1, V_3), (V_2, V_4)\}$
 5. *Kiểm tra* $(V_4, V_5; 4)$: Thêm vào, tạo ra chu trình $V_1 - V_2 - V_4 - V_5 - V_3 - V_1$. **Bỏ qua.**
- ❸ Kết quả
 - Cây khung nhỏ nhất gồm các cạnh:
 $(V_1, V_2), (V_3, V_5), (V_1, V_3), (V_2, V_4)$.
 - Tổng trọng số: $1 + 1 + 2 + 3 = 7$.



Thuật toán Kruskal tìm MST

Tính đúng đắn của thuật toán, gồm 2 phần: (1) kết quả thuật toán là một cây bao trùm; (2) cây bao trùm đó là nhỏ nhất.

❶ **Cây bao trùm:** F luôn là một rừng do việc nối hai cây bằng một cạnh luôn tạo ra một cây mới. Giả thiết phản chứng, F gồm ít nhất hai cây A và B . Khi cạnh đầu tiên nối các đỉnh trong A của F với phần còn lại của đồ thị được xem xét (cạnh này tồn tại do G liên thông) thì thuật toán sẽ chọn nó. $\Rightarrow A$ không thể là một cây trong F khi thuật toán kết thúc. $\Rightarrow F$ liên thông và là một cây bao trùm.

❷ **Nhỏ nhất:** Chứng minh mệnh đề P sau đây **bằng quy nạp**: "Nếu F là tập hợp các cạnh đã chọn tại bất kì thời điểm nào trong quá trình thực hiện thuật toán, thì tồn tại cây bao trùm nhỏ nhất chứa F ".

- P đúng khi thuật toán bắt đầu, vì $F = \emptyset$.
- Giả sử P là đúng đối với một tập hợp F và giả sử T là MST chứa F . **Nếu cạnh được thêm vào tiếp theo là e cũng nằm trong T** , thì P đúng đối với $F + e$. **Nếu không**, thì $T + e$ chứa chu trình C và tồn tại cạnh f nằm trên C nhưng không trong F . (Nếu không có cạnh f , thì không thể thêm e vào F , do sẽ tạo ra chu trình C trong F .) Do đó $T - f + e$ là một cây, và nó có cùng trọng số với T , do T có trọng số nhỏ nhất và f không thể nhỏ hơn e , vì nếu không thuật toán đã xem xét f trước e và chọn f . Vì vậy $T - f + e$ là một cây bao trùm nhỏ nhất chứa $F + e$ và P là đúng.
- Như vậy, P đúng khi thuật toán kết thúc và F là một cây bao trùm. Điều này chỉ có thể xảy ra nếu F là một cây bao trùm nhỏ nhất.

Thuật toán Kruskal tìm MST

Độ phức tạp thời gian

- Nếu E là số cạnh và V là số đỉnh của đồ thị thì thuật toán Kruskal chạy trong thời gian $O(E \log E)$ hoặc $O(E \log V)$.

Sự khác nhau giữa hai thuật toán Prim và Kruskal

- ❶ **Cách tiếp cận:** **Prim** bắt đầu từ một **đỉnh** và phát triển cây ra ngoài, phù hợp đồ thị dày (**nhiều cạnh**). **Kruskal** sắp xếp tất cả các **cạnh** và thêm dần, phù hợp đồ thị thưa (*ít cạnh*).
- ❷ **Cấu trúc dữ liệu:** **Prim** thường dùng danh sách kề và hàng đợi ưu tiên (Priority Queue/Heap). **Kruskal** dùng danh sách cạnh và cấu trúc Union-Find (Disjoint Set) để kiểm tra chu trình.
- ❸ **Phạm vi hoạt động:** **Prim** phát triển một cây duy nhất. **Kruskal** có thể tạo ra một "rừng" (nhiều cây) và sau đó kết nối chúng lại.

Section 6

Thuật toán Dijkstra tìm đường đi ngắn nhất

Thuật toán Dijkstra tìm đường đi ngắn nhất

- **Bài toán:** Cho Đồ thị vô hướng có trọng số và 1 đỉnh nguồn. Tìm đường đi ngắn nhất từ đỉnh nguồn đến tất cả các đỉnh trong Đồ thị.

- **Thuật toán Dijkstra:**

- ➊ **Khởi tạo:** Đánh dấu các đỉnh

- a Đỉnh nguồn = 0;

- b Các đỉnh khác nguồn = ∞ .

- ➋ **Đỉnh chưa xét** với **giá trị đánh dấu nhỏ nhất**, gọi là C (Current);

- ➌ Với mỗi đỉnh kề với đỉnh C (gọi là đỉnh N)

- a d = Giá trị đánh dấu của đỉnh C + Trọng số của cạnh (C,N);

- b Nếu $d <$ giá trị đánh dấu của đỉnh N, thì cập nhật giá trị cho đỉnh N bởi d.

- ➍ Đánh dấu đỉnh C đã xét (loại bỏ C)

- ➎ Nếu còn đỉnh chưa xét thì lặp lại Bước 2.

- **Giải mã:**

Dijkstra($G(V, E)$, *source*)

$dist[source] := 0;$

for each $v \in G$

- **if** $v \notin source$, **then** $dist[v] := \infty;$

- add v to Q

while $Q \neq \emptyset$

- xác định $v \in Q$ với $dist[v] = \min$ // lần đầu, v là đỉnh *source*

- $Q = Q \setminus \{v\}$

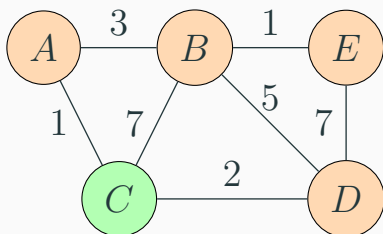
- **for** each lân cận N của v // không bao gồm các đỉnh đã bị loại khỏi Q
 $dist[N] := \min(dist[N], dist[v] + length(v, N))$

return $dist[]$

endfunction

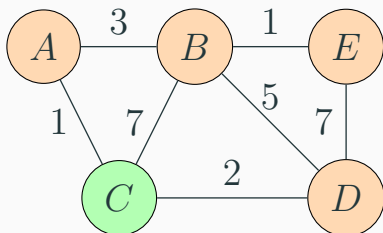
Thuật toán Dijkstra tìm đường đi ngắn nhất

Ví dụ minh họa: Cho Đồ thị vô hướng có trọng số G và đỉnh nguồn C

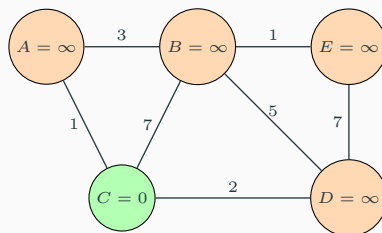


Thuật toán Dijkstra tìm đường đi ngắn nhất

Ví dụ minh họa: Cho Đồ thị vô hướng có trọng số G và đỉnh nguồn C

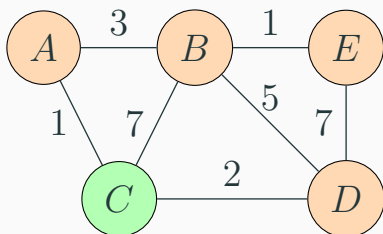


Khởi tạo

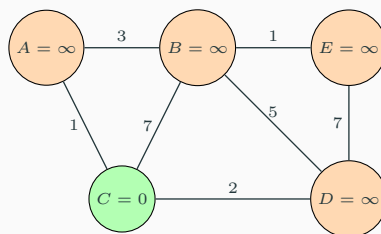


Thuật toán Dijkstra tìm đường đi ngắn nhất

Ví dụ minh họa: Cho Đồ thị vô hướng có trọng số G và đỉnh nguồn C

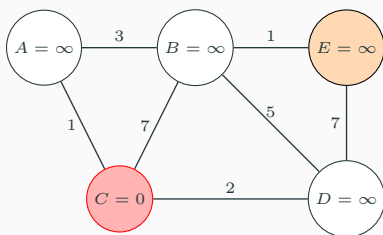


Khởi tạo



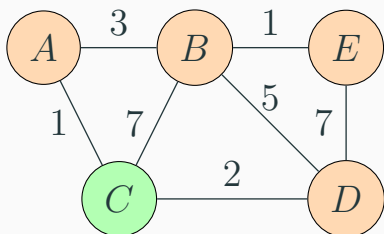
Ban đầu: xét đỉnh nguồn (C)

Các đỉnh lân cận: A, B, D

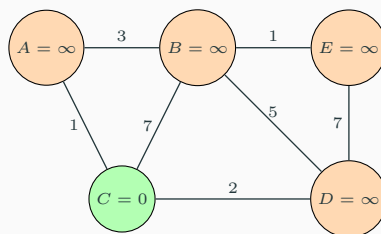


Thuật toán Dijkstra tìm đường đi ngắn nhất

Ví dụ minh họa: Cho Đồ thị vô hướng có trọng số G và đỉnh nguồn C

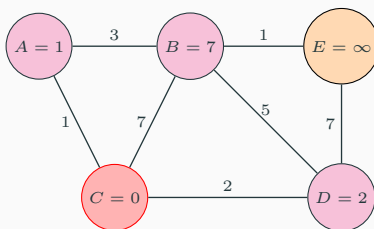
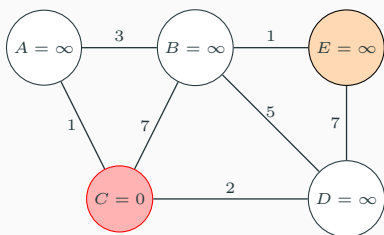


Khởi tạo



Ban đầu: xét đỉnh nguồn (C)

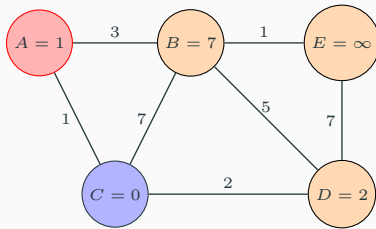
Các đỉnh lân cận: A, B, D



Thuật toán Dijkstra tìm đường đi ngắn nhất

Tiếp: Xét A (chưa xét, min)

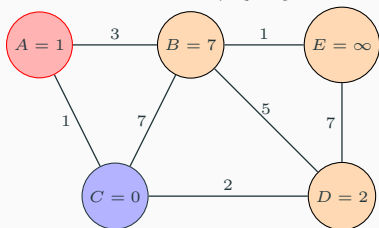
$$Q = Q \setminus \{C\}$$



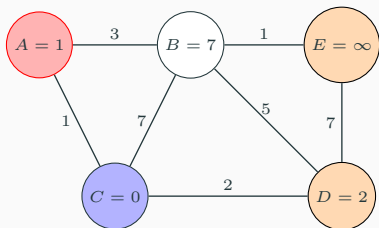
Thuật toán Dijkstra tìm đường đi ngắn nhất

Tiếp: Xét A (chưa xét, min)

$$Q = Q \setminus \{C\}$$



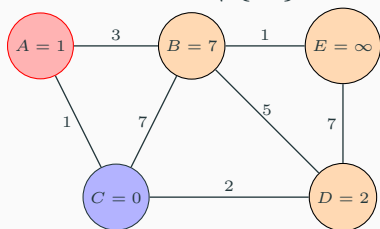
Lân cận: B



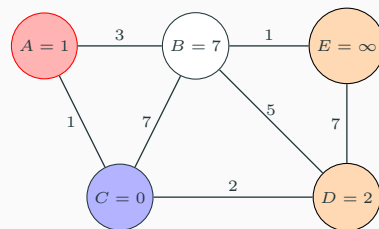
Thuật toán Dijkstra tìm đường đi ngắn nhất

Tiếp: Xét A (chưa xét, min)

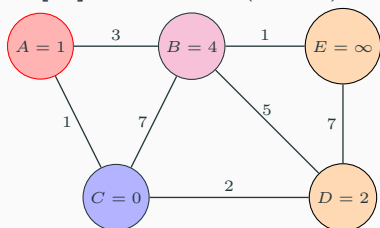
$$Q = Q \setminus \{C\}$$



Lần cận: B



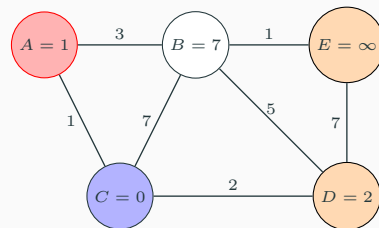
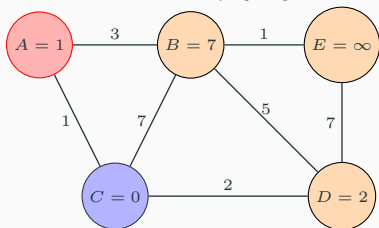
$$\text{dist}[A] + \text{length}(A, B) = 4$$



Thuật toán Dijkstra tìm đường đi ngắn nhất

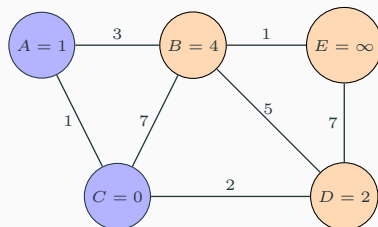
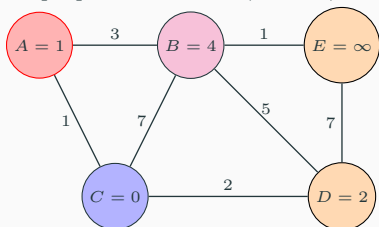
Tiếp: Xét A (chưa xét, min)

$$Q = Q \setminus \{C\}$$



Lần cận: B

$$\text{dist}[A] + \text{length}(A, B) = 4$$

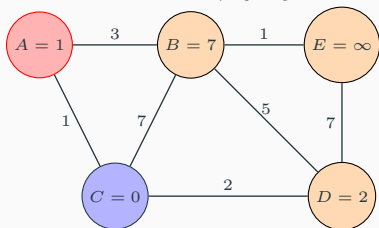


$$Q = Q \setminus \{A\}$$

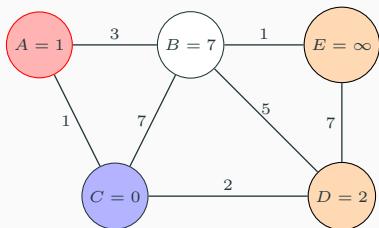
Thuật toán Dijkstra tìm đường đi ngắn nhất

Tiếp: Xét A (chưa xét, min)

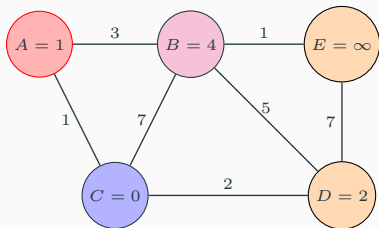
$$Q = Q \setminus \{C\}$$



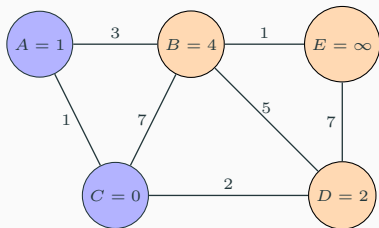
Lần cận: B



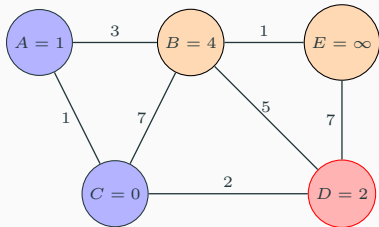
$$\text{dist}[A] + \text{length}(A, B) = 4$$



$$Q = Q \setminus \{A\}$$



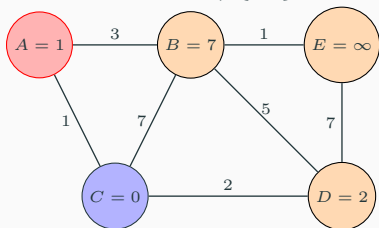
Lặp lại thuật toán, xét D



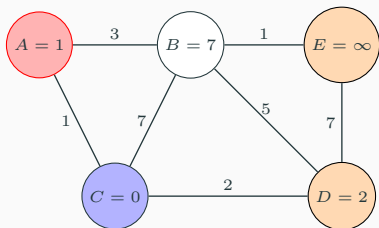
Thuật toán Dijkstra tìm đường đi ngắn nhất

Tiếp: Xét A (chưa xét, min)

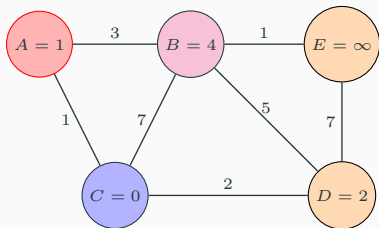
$$Q = Q \setminus \{C\}$$



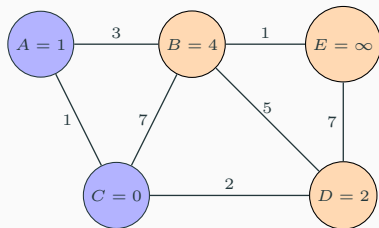
Lân cận: B



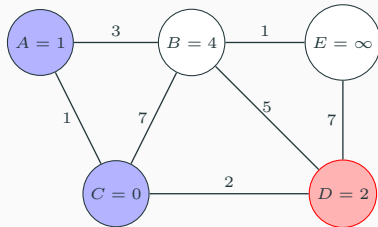
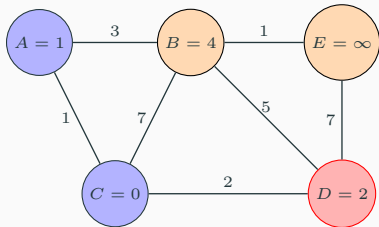
$$\text{dist}[A] + \text{length}(A, B) = 4$$



$$Q = Q \setminus \{A\}$$



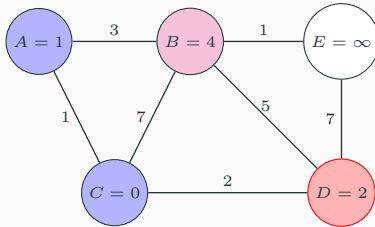
Lặp lại thuật toán, xét D



Kề với D là B và E

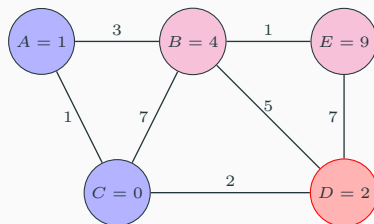
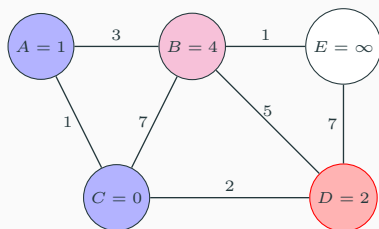
Thuật toán Dijkstra tìm đường đi ngắn nhất

Xét B, $\text{dist}[D] + \text{length}(D, B)$
 $= 2 + 5 > 4 \Rightarrow \text{NO update}$

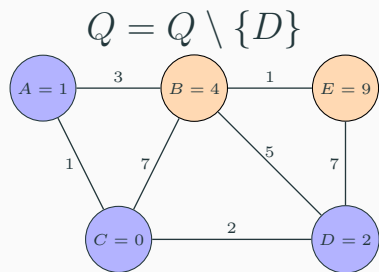


Thuật toán Dijkstra tìm đường đi ngắn nhất

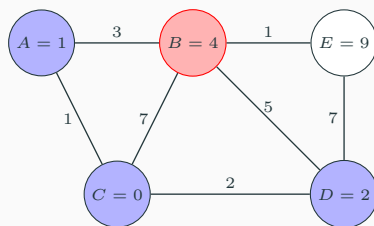
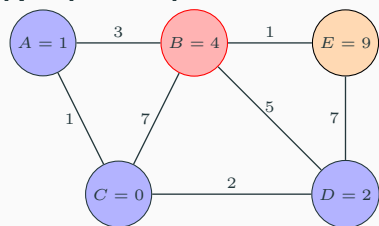
Xét B, $\text{dist}[D] + \text{length}(D,B)$
 $= 2 + 5 > 4 \Rightarrow \text{NO update}$



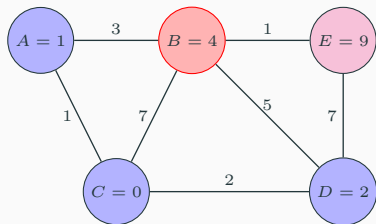
Xét E, $\text{dist}[D] + \text{length}(D,E)$
 $= 2 + 7 < \infty \Rightarrow \text{update } E = 9$



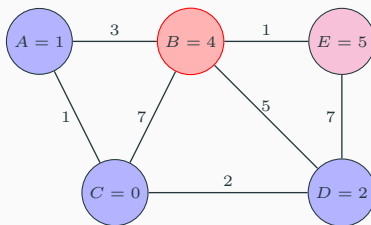
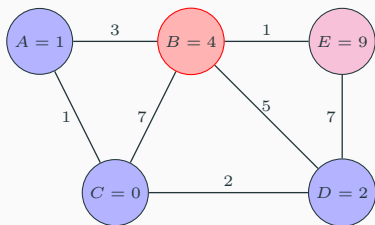
Lặp lại thuật toán, xét B



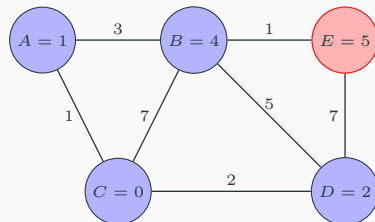
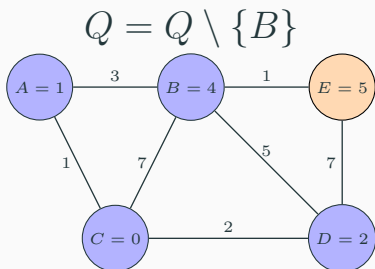
Thuật toán Dijkstra tìm đường đi ngắn nhất



Thuật toán Dijkstra tìm đường đi ngắn nhất



$$\begin{aligned} & \text{dist}[B] + \text{length}(B, E) \\ &= 4 + 1 < 9 \Rightarrow \text{update } E = 5 \end{aligned}$$

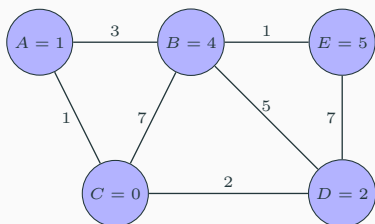


chuyển E

E không có đỉnh kề nào chưa xét

$\Rightarrow$ đánh dấu nốt đỉnh E

$$Q = Q \setminus \{E\}$$



Kết luận

$C \rightarrow A : 1$

$C \rightarrow B : 4$

$C \rightarrow D : 2$

$C \rightarrow E : 5.$

Section 7

Bài tập

- Program the designed algorithms
- Other examples or application